

Ciência de Dados Aplicada

Aula 16 — Projeto AV2: Diretrizes, Entregáveis e Formação dos Grupos

Mateus Silva

Faculdade de Petrolina

Objetivos desta Aula

Ao final desta aula, você será capaz de:

- Compreender o escopo completo e os critérios de avaliação do Projeto AV2.
- Identificar os três entregáveis obrigatórios: repositório, artigo e apresentação.
- Escolher um problema de dados real adequado ao nível da disciplina.
- Aplicar o CRISP-DM como guia metodológico do projeto.
- Formar seu grupo e definir o tema antes do encerramento da aula.

De Onde Viemos — Para Onde Vamos

AV1 — Aulas 1 a 15:

- Pensamento analítico e CRISP-DM
- Pipelines reprodutíveis e leakage
- Batch vs. Streaming
- Drift e monitoramento em produção
- Prova de síntese (Aula 15)

AV2 — Aulas 16 a 30:

- **Hoje:** Kick-off do Projeto AV2
- Aulas 17–22: Conteúdo avançado em paralelo
- Aula 23: Checkpoint do projeto (EDA + pipeline)
- Aulas 24–28: Finalização e ensaio
- Aulas 29–30: **Defesas finais**

O que é o Projeto AV2?

Definição

O Projeto AV2 é um trabalho em grupo que simula um projeto real de Ciência de Dados, seguindo a metodologia CRISP-DM do entendimento do problema até o monitoramento em produção.

Características gerais:

- Grupos de **3 a 4 pessoas**
- Problema real com dados reais (Kaggle, UCI, dados abertos, etc.)
- Modelo treinado e avaliado com rigor — sem leakage
- Código versionado, reproduzível e containerizado
- Monitoramento e observabilidade presentes no sistema

Atenção

Não é um notebook isolado. É um projeto de software com dados, modelo e infraestrutura.

Peso e Composição da Nota

Entregável	Peso no AV2	Como entregar
Repositório GitHub	50%	Link público ao final do prazo
Artigo (máx. 5 páginas)	30%	PDF no repositório
Apresentação (5 slides)	20%	PDF no repositório + defesa presencial

Nota final da disciplina:

$$\text{Nota Final} = 0,50 \times \text{AV1} + 0,50 \times \text{AV2}$$

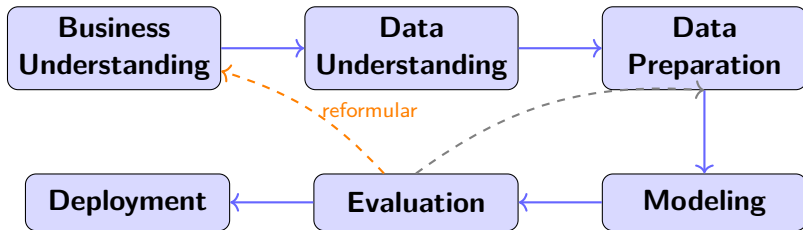
CRISP-DM: Espinha Dorsal do Projeto

O projeto deve seguir **e documentar** todas as fases do CRISP-DM:

- ① **Business Understanding** — Problema, métrica de sucesso, stakeholder hipotético
- ② **Data Understanding** — EDA, distribuições, qualidade dos dados, anomalias
- ③ **Data Preparation** — Limpeza, feature engineering, splits corretos (sem leakage)
- ④ **Modeling** — Escolha do modelo, treino, comparação de experimentos
- ⑤ **Evaluation** — Métricas alinhadas ao problema, análise de erros por segmento
- ⑥ **Deployment** — Containerização, API ou pipeline agendado, monitoramento

Cada fase deve ter rastro no repositório e aparecer no artigo.

Fluxo CRISP-DM Aplicado ao Projeto



Setas tracejadas indicam retornos iterativos — esperados e documentados no artigo como decisões técnicas.

Entregável 1 — Repositório GitHub (50%)

Estrutura mínima obrigatória:

- README.md — Descrição, como rodar, resultado principal
- pyproject.toml — Dependências Python gerenciadas
- Dockerfile — Containerização da solução
- data/ — Scripts de aquisição (não o dado bruto grande)
- notebooks/ — EDA exploratório inicial
- src/ — Código de pipeline, treino e serving
- reports/ — Artigo PDF e slide PDF

Boas práticas:

- Commits com mensagens claras
- .gitignore adequado
- Variáveis sensíveis em .env (não comitar)
- Tag de versão na entrega final (v1.0)
- Makefile ou script de setup facilita avaliação

Arquivo Obrigatório: `pyproject.toml`

Por que usar `pyproject.toml`?

Padrão moderno (PEP 517/518) que define dependências, versão do Python e metadados. Garante reprodutibilidade do ambiente sem ambiguidade.

Arquivo Obrigatório: Dockerfile

Por que containerizar?

Garante que qualquer avaliador execute o projeto em qualquer máquina, sem instalar nada manualmente. É o padrão da indústria para deploy de modelos.

```
FROM python:3.11-slim

WORKDIR /app
COPY pyproject.toml .
RUN pip install --no-cache-dir .

COPY src/ ./src/
COPY models/ ./models/

EXPOSE 8000
CMD ["uvicorn", "src.api:app", "--host", "0.0.0.0", "--port", "8000"]
```

Entregável 2 — Artigo Científico (30%, máx. 5 páginas)

Estrutura esperada:

- 1 **Introdução** — Problema, motivação e objetivo (0,5 pág.)
- 2 **Dados** — Fonte, descrição, EDA resumido (1 pág.)
- 3 **Metodologia** — Pipeline, features, modelo, validação (1,5 pág.)
- 4 **Resultados** — Métricas, análise de erros, comparação de experimentos (1 pág.)
- 5 **Conclusão e Trabalhos Futuros** — Limitações e próximos passos (0,5 pág.)
- 6 **Referências** — Dados, papers, frameworks utilizados

Dica de escrita

Escreva como se o leitor não tivesse cursado a disciplina: defina termos, explique escolhas e conecte métricas ao problema de negócio.

Entregável 3 — Apresentação de 5 Slides (20%)

Cada grupo terá **10 minutos** de apresentação + 5 minutos de perguntas.

Estrutura dos 5 slides:

- 1 **Problema e contexto de negócio** — Quem se beneficia? Qual a decisão suportada?
- 2 **Dados** — Fonte, volume, EDA (1–2 gráficos mais relevantes)
- 3 **Pipeline e decisões técnicas** — Modelo escolhido, features, validação
- 4 **Resultados** — Métricas com interpretação de negócio (não só o número)
- 5 **Arquitetura de produção** — Orquestração + monitoramento + como o sistema vive

Os slides contam uma história: **problema → solução → resultado → impacto.**

Fontes de Dados Recomendadas

Fontes principais:

- **Kaggle** kaggle.com/datasets — ampla variedade, competições reais
- **UCI ML Repository**
archive.ics.uci.edu — clássicos bem documentados
- **Dados Abertos Brasil**
dados.gov.br — saúde, educação, finanças públicas
- **IBGE / BCB / ANS** — datasets nacionais de qualidade

Critérios para escolher bem:

- Problema real com interpretação de negócio
- Volume razoável: ≥ 1.000 registros
- Features interessantes para engenharia
- Target bem definido e justificável
- Licença permite uso acadêmico

Evite

Datasets brinquedo sem contexto real (ex.: Iris isolado, Titanic sem análise crítica).

Ferramentas de Orquestração — Opcional, Valorizado

Ferramentas de orquestração automatizam e sequenciam etapas do pipeline de dados e modelo.

Apache Airflow

- DAGs em Python
- Vasto ecossistema
- Mais complexo de configurar

Kestra

- YAML-first
- UI moderna e intuitiva
- Ótimo para protótipos

n8n

- No-code/low-code
- Ideal para integração de APIs
- Bom para coleta de dados

Qualquer ferramenta é válida. O importante é que ela apareça na arquitetura com propósito claro e seja documentada no README.md.

Ferramentas de Monitoramento — Obrigatório

O projeto **deve** incluir alguma forma de observabilidade do modelo ou pipeline.

MLflow (recomendado)

- Registro de experimentos e hiperparâmetros
- Versionamento de modelos
- UI de comparação de runs
- Integra com praticamente tudo

Evidently AI

- Detecção automática de data drift
- Relatórios HTML prontos
- Integra com MLflow

Grafana + Prometheus

- Dashboards em tempo real
- Alertas configuráveis
- Ideal para monitorar APIs

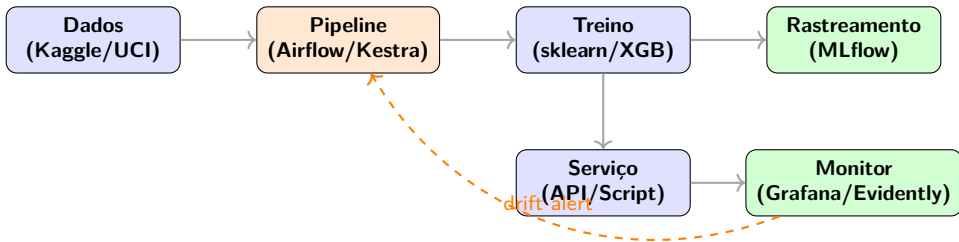
Weights & Biases (W&B)

- Similar ao MLflow
- Boa UX, tier gratuito

Mínimo aceitável

MLflow ou W&B rastreando ao menos 5 experimentos comparados.

Arquitetura de Referência do Projeto



Adapte ao seu projeto — a orquestração é opcional, mas o monitoramento é obrigatório.

Exemplo 1 — Detecção de Fraude em Cartão de Crédito

Problema: Identificar transações fraudulentas em tempo quase real.

Dados: Kaggle — *Credit Card Fraud Detection*

Desafios técnicos:

- Desbalanceamento extremo de classes
- Custo assimétrico: falso negativo custa muito mais
- Validação temporal obrigatória

O que explorar:

- SMOTE e `class_weight`
- Threshold calibration por custo
- AUC-PR como métrica principal
- Monitorar PSI das features ao longo do tempo

Complexidade: 3/5

Exemplo 2 — Churn em Telecomunicações

Problema: Prever quais clientes irão cancelar o serviço nos próximos 30 dias.

Dados: Kaggle — *Telco Customer Churn* (IBM)

Desafios técnicos:

- Features categóricas e numéricas mistas
- Definir o horizonte de predição (janela de tempo)
- Custo assimétrico de retenção vs. perda de cliente

O que explorar:

- XGBoost vs. Regressão Logística
- SHAP para explicar quais features mais pesam
- Pipeline sklearn completo
- Simulação de impacto financeiro da intervenção

Complexidade: 2/5

Exemplo 3 — Previsão de Inadimplência de Crédito

Problema: Prever se um tomador de empréstimo irá inadimplir nos próximos 90 dias.

Dados: UCI — *Default of Credit Card Clients*

Desafios técnicos:

- Feature engineering em séries de pagamentos mensais
- Risco alto de target leakage em variáveis de saldo
- Validação out-of-time rigorosa

O que explorar:

- Scorecard com pontuação interpretável
- Drift do score ao longo do tempo
- Análise de erro por faixa de renda

Complexidade: 4/5

Exemplo 4 — Qualidade de Produtos (Indústria ou Alimentação)

Problema: Prever a qualidade sensorial de vinhos a partir de propriedades físico-químicas — ou adaptável a controle de qualidade industrial.

Dados: UCI — *Wine Quality*

Desafios técnicos:

- Target ordinal — classificação ou regressão?
- Multicolinearidade entre features químicas
- Bins de qualidade: onde cortar?

O que explorar:

- Random Forest com feature importance
- Análise de resíduos em regressão ordinal
- Monitoramento de distribuição do target ao longo das colheitas
- Comparação de 3+ modelos no MLflow

Complexidade: 2/5

Exemplo 5 — Diagnóstico Médico

Problema: Diagnosticar doença a partir de dados clínicos — diabetes, doença cardíaca ou câncer de mama.

Dados (escolha um):

- UCI — *Heart Disease*
- Kaggle — *Pima Indians Diabetes*
- UCI — *Breast Cancer Wisconsin*

Desafios técnicos:

- Alta sensibilidade ao falso negativo (vida em risco)
- Explicabilidade obrigatória em contexto clínico

O que explorar:

- SHAP / LIME para interpretabilidade
- Curva ROC e AUC por subgrupo demográfico
- Threshold otimizado por custo clínico

Complexidade: 3/5

Exemplo 6 — Previsão de Demanda em Varejo

Problema: Prever volume de vendas por produto e loja para otimizar estoque.

Desafios técnicos:

- Séries temporais com sazonalidade e feriados
- Validação temporal: expanding window
- Escolha de métrica: RMSE vs. MAPE (por produto)

O que explorar:

- Features de calendário e lags
- LightGBM para séries tabulares
- Monitoramento de drift das séries
- Impacto financeiro do erro de previsão

Complexidade: 4/5

Critérios de Avaliação do Projeto

Critério	Peso	Onde aparece
Formulação do problema (Business Understanding)	10%	Artigo + Apresentação
Qualidade e rastreabilidade do pipeline	20%	Repositório
Rigor da modelagem e validação (sem leakage)	20%	Repositório + Artigo
Observabilidade (MLflow + outros)	15%	Repositório
Reprodutibilidade (Docker + pyproject.toml)	15%	Repositório
Qualidade do artigo (clareza, estrutura, rigor)	10%	Artigo
Clareza e profundidade da defesa	10%	Defesa

Armadilhas Comuns — Evite no Projeto

Armadilhas técnicas:

- Aplicar `fit_transform` no conjunto de teste
- Não versionar o modelo treinado (somente o notebook)
- Usar acurácia em dataset desbalanceado
- Esquecer de serializar o scaler junto com o modelo
- Não definir a versão do Python e das bibliotecas

Armadilhas de processo:

- Modelar antes de entender o problema
- Apresentar resultado sem análise de erro
- Docker e monitoramento deixados para a última hora
- Artigo escrito com pressa após a apresentação
- Todos do grupo editam o mesmo arquivo sem controle de versão

O que Diferencia um Projeto Nota 10

Projeto mediano

Treina um modelo, reporta acurácia ou F1, tem um repositório com notebook. Docker não funciona sem ajustes. MLflow não aparece.

Projeto nota 10

- Problema formulado com critério de negócio real — quem decide e com base em quê?
- Validação out-of-time, sem vazamento, com análise de erro por segmento
- `docker build && docker run` funciona em qualquer máquina sem ajustes
- MLflow com pelo menos 5 experimentos comparados e modelo registrado
- Artigo conta uma história técnica com começo, meio e fim
- Apresentação conecta resultado técnico com impacto de negócio estimado
- Monitoramento detecta e alerta sobre drift — não apenas registra logs

Formação dos Grupos

Regras

- Grupos de **3 a 4 integrantes**
- Cada grupo define um **líder técnico** responsável pelo repositório
- Grupos **devem ser formados hoje**, antes do encerramento da aula
- Temas devem ser definidos em até **uma semana**.

Definições obrigatórias até o final desta aula:

- 1 Nome completo de todos os integrantes
- 2 Tema e problema escolhido (ou top 2 candidatos)
- 3 Fonte de dados confirmada (link do dataset)
- 4 URL do repositório GitHub criado (pode ser privado por enquanto)

Cronograma do Projeto AV2

Aula	Marco	Entrega / Atividade
16	Kick-off	Grupo e tema definidos; repositório criado
17–22	Desenvolvimento	Conteúdo avançado + projeto em paralelo
23	Checkpoint	EDA completo + pipeline base no repositório (feedback do professor)
24–27	Finalização	Modelo, monitoramento, artigo e slides
28	Ensaio	Apresentação interna do grupo (10 min + feedback)
29–30	Defesa Final	Apresentações para a turma + submissão do repositório